

SYIR - Systemy Internetu Rzeczy

Opis modułów wykorzystywanych podczas LAB 1

Arkadiusz Łuczyk, Jakub Jasiński, Marek Niewiński

Politechnika Warszawska

Instytut Mikroelektroniki i Optoelektroniki

Zakład Metod Projektowania w Mikroelektronice

syirDevice_CLOCKS

Moduł do ustawiania źródła i częstotliwości urządzenia

oraz okresu zegara systemowego (ang. *system tick*)

```
typedef enum {...} syirDevice_DeviceClockSource_t;
```

Typ wyliczeniowy wyboru źródła sygnału zegara urządzenia

```
typedef enum {...} syirDevice_DeviceClockDivision_t;
```

Typ wyliczeniowy podziału sygnału zegara urządzenia

```
typedef enum {...} syirDevice_SystemClockDivision_t;
```

Typ wyliczeniowy podziału zegara urządzenia by utworzyć zegar systemowy

```
typedef uint16_t systemClock_t;
```

Typ dla zmiennych przechowujących stan zegara systemowego

```
int8_t syirDevice_SET_deviceClock(src , div );
```

syirDevice_CLOCKS (kont.)

Ustawia źródło zegara dla urządzenia (argument pierwszy funkcji) oraz krotności podziału częstotliwości źródła (drugi argument funkcji) aby ustalić częstotliwość pracy urządzenia

Funkcja zwraca wartość > 0 gdy pomyślnie się wykona; w przeciwnym razie zwraca -1

```
int8_t syirDevice_SET_systemClock (period, ckDiv);
```

Ustawia liczbę zliczeń licznika **TCC0** (pierwszy argument) oraz krotność podziału zegara urządzenia aby utworzyć zegar systemowy

Licznik **TCC0** po osiągnięciu zadanej wartości period spowoduje wywołanie procedury `ISR(TCC0_OVF_vect)`

Funkcja zwraca > 0 jeśli gdy pomyślnie się wykona; w przeciwnym razie zwraca -1

```
int8_t syirDevice_CHECK_systemClock(systemClock_t *prev);
```

Funkcja nieblokująca zwraca 1 gdy jej poprzednie wywołanie było dawniej niż jeden takt zegara systemowego. W przeciwnym wypadku zwraca 0

syirDevice_CLOCKS (kont.)

Argumentem funkcji jest wskaźnik do zmiennej przechowującej ostatnią zmianę zegara systemowego

```
int8_t syirDevice_CHECK_N_systemClock(uint16_t N, systemClock_t *prev);
```

Funkcja nieblokująca zwraca 1 gdy jej poprzednie wywołanie było dawniej niż N taktów zegara systemowego. W przeciwnym wypadku zwraca 0

Pierwszym argumentem funkcji jest liczba N taktów zegara systemowego. Drugim argumentem funkcji jest wskaźnik do zmiennej przechowującej ostatnią zmianę zegara systemowego

syirDevice_CLOCKS (kont.)

```
typedef struct { } syirDevice_StatusSystemClock_t;
```

Typ dla zmiennej stanu zegara systemowego

```
syirDevice_StatusSystemClock_t statusCLOCKS
```

Zmienna lokalna stanu zegara systemowego

```
statusCLOCKS.tick
```

Przechowuje aktualny stan zegara systemowego i jest modyfikowany przez procedurę obsługi przerwania `ISR(TCC0_OVF_vect)`

syirDevice_LOG

Moduł realizujący komunikację po UART w celu wysyłania komunikatów tekstowych odbieranych przez YAT

Komunikacja wykorzystuje nadawanie bajtów wbudowanym kontrolerem **UARTC0**

```
void syirDevice_INIT_log(void);
```

Funkcja inicjalizuje zmienną lokalną stanu modułu `statusLog`; ustawia odpowiednio piny portu **PORTC** do komunikacji UART oraz tryb pracy kontrolera **UARTC** (zobacz dokumentację kontrolera UART oraz portów PORT oraz ich rejestrów konfiguracyjnych w manualu mikrokontrolera ATXMEGA256A3BU)

```
int8_t syirDevice_SET_log(int8_t BSCALE, uint16_t BSEL, uint8_t CLK2X);
```

Funkcja konfiguruje kontroler **UARTC0** i jego podzielniki częstotliwości urządzenia taki aby uzyskać zadany baudrate w komunikacji UART

Parametry funkcji są wyznaczone przez skrypt pythonowy **get_uart_settings** którego argumentami są parametry ustalane w funkcji

syirDevice_LOG (kont.)

`int8_t syirDevice_SET_deviceClock (...)`; określającej częstotliwość pracy urządzenia

Jeśli podane wartości parametrów funkcji są poza zakresami to funkcja zwraca wartość < 0 ; w przeciwnym wypadku zwraca 1

`int8_t syirDevice_SEND_log (char *buf, uint16_t num)`;

Funkcja jest nieblokująca i jako argument otrzymuje wskaźnik do tekstu oraz jego rozmiar który następnie w kolejnych stanach i przebiegach funkcji jest wysyłany przez UART

Stan jest zmieniany za każdym wysłaniem znaku w procedurze `ISR(USARTC0_TXC_vect)`

syirDevice_REC

Moduł realizujący komunikację po UART w celu odbierania komunikatów tekstowych z YAT

Komunikacja wykorzystuje odbieranie bajtów wbudowanym kontrolerem **UARTC0**

```
void syirDevice_INIT_rec(void);
```

Funkcja inicjalizuje zmienną lokalną stanu modułu `statusRec`; ustawia odpowiednio piny portu **PORTC** do komunikacji UART oraz tryb pracy kontrolera **UARTC** (zobacz dokumentacje kontrolera UART oraz portów PORT oraz ich rejestrów konfiguracyjnych w manualu mikrokontrolera ATXMEGA256A3BU)

```
int8_t syirDevice_GET_rec (char *buf);
```

Funkcja jest nieblokująca i jako argument otrzymuje wskaźnik do bufora znaków. Jeśli UARTC odbierze pojedynczy znak to wstawiany jest on do bufora **buf** i funkcja zwraca wartość 1. W przeciwnym wypadku zwraca wartość 0.

Stan jest zmieniany za każdym gdy odebrany jest znak w procedurze `ISR (USARTC0_RXC_vect)`

syirDevice_PORTS

Moduł zawiera funkcje obsługi portów mikrokontrolera

```
void syirDevice_OFF_ports(void);
```

Funkcja wyłącza wszystkie porty - ustawia ich kierunek na wejściowy tak aby nie zasilaly z pinu żadnego urządzenia zawnętrznego podłączonego do mikrokontrolera

syirDevice_LEDS

Plik nagłówkowy zawiera makrodefinicje strujące pinami portu **PORTF** do których podłączone są diody LED: **GREEN** i **ORANGE**

```
syirDevice_ON_led(led);
```

Włączenie diody led

```
syirDevice_OFF_led(led);
```

Wyłączenie diody led

```
syirDevice_TOGGLE_led(led);
```

Przełączenie stanu diody